

Integration Guide

Scientific Data Analysis v1.8

Backend contracts for app integration

App invocation, run bundles, dry-run routing, and explicit limits

English publication edition: September 23, 2026

Translation of the preserved private v1.8 guide at commit 994eaea. Original TeX SHA-256: 41c5839931276961b04a6377cd247591fcf30940f120361b9b95a9b8583efdcf. Private source: `skills/scientific-data-maintainer/.cache/active_references/guides/guia_scientific_data_analysis_v1.8.tex`.

This guide records historical backend preparation, not current verification or installed-family promotion. Its v1.8 scope does not modify or synchronize Scientific Workbench. Historical commands are illustrative: use fresh output directories when reproducing them without overwriting earlier evidence. The example task prompt is translated into English; script names, options, and contract identifiers are preserved. The original remains unchanged privately.

Abstract

`scientific-data-analysis` v1.8 is a contract release. It prepares the editable skill as a more predictable backend for Scientific Workbench, without claiming that the app is already synchronized to v2.0. The backend comes first: parseable JSON, artifact types, run bundles, dry-run routing, an app-readiness matrix, app-like regressions, and explicit separation of normal, expert, optional, legacy, maintainer, and unexposed modes.

v1.7 asked whether the skill was useful in real settings. v1.8 asks whether an app can invoke it predictably, read the result, present artifacts, and explain the next step while preserving original inputs.

Contents

1	Executive overview	1
2	From v1.7 to v1.8	1
3	What remains unchanged	1
4	The v1.8 JSON contract	2
4.1	Canonical statuses	2
5	Artifact types	2
6	Run bundles	3
7	Dry-run router	3
8	Capability app-readiness	4
8.1	Recommended exposure	4
9	Testing a capability as an app would	4
10	Optional, legacy, and domain-specific routes	5
10.1	Optional	5
10.2	Legacy	5
10.3	Domain-specific	5
11	Scientific Workbench	5
12	v1.8 regressions	5
13	Work reserved for v1.9	5
14	Work reserved for v2.0	6
15	Historical Phase 8 checklist	6

1. Executive overview

v1.8 prepares the skill as a Scientific Workbench backend. It does not develop or synchronize the app, move it to v2.0, or promise that the app already consumes every new field. The backend changes are:

- A v1.8 JSON contract for app-ready outputs.
- An initial artifact type taxonomy.
- A stable run bundle format.
- An app-readiness matrix for all 59 public capabilities.
- A deterministic dry-run router.
- App-like regressions for general, scientific, astronomy, optional, and legacy routes.
- Integration guidance that reserves major app synchronization for v2.0.

Product intent: each capability should state whether it applies, warns, blocks, or fails cleanly, which artifacts it produced, and what a person should do next. v1.8 does not make every capability general-purpose.

2. From v1.7 to v1.8

Area	v1.8 change
Contract	A JSON envelope with <code>contract_version</code> , <code>app_status</code> , <code>typed_artifacts</code> , <code>original_modified</code> , <code>warnings</code> , <code>errors</code> , <code>next_actions</code> , and <code>app_hints</code> .
App-readiness	All 59 capabilities are classified as <code>app_ready</code> , <code>app_ready_partial</code> , <code>cli_only</code> , <code>blocked_optional</code> , <code>maintainer_only</code> , or <code>not_applicable_to_app</code> .
Run bundles	A stable run folder with <code>summary.json</code> , <code>manifest.json</code> , logs, artifacts, previews, reports, tables, and <code>next_steps.md</code> .
Router	<code>scientific_workflow_router.py</code> is a conservative dry-run planner. It recommends or rejects routes without executing destructive actions.
App-like regressions	General, scientific, astronomy, optional, and legacy families and bundles are exercised as a local app would invoke them.
Integration	Guidance explains how Scientific Workbench can use the contracts later, without claiming v2.0 integration.

3. What remains unchanged

- The installed skill remains at its applicable stable version until final synchronization.
- Scientific Workbench is not edited in this phase.
- Domain-specific capabilities retain their domain limits.
- Optional routes do not become core dependencies.
- Legacy routes are not ordinary user actions.
- Original user inputs remain read-only by default.
- v1.8 builds on the real-world foundation of v1.7.

4. The v1.8 JSON contract

Primary reference: `references/v1-8-app-ready-contract.md`.

Minimum fields expected in app-ready output:

Field	Use
<code>contract_version</code>	1.8 for an app-ready envelope.
<code>tool</code>	Stable script or route name, retained for compatibility.
<code>capability_id</code> / <code>capability_label</code>	Registry identity where applicable.
<code>status</code> / <code>app_status</code>	Canonical or normalized status.
<code>command</code>	<code>argv</code> , <code>cwd</code> , and redaction. Must never expose secrets.
<code>inputs</code>	Inspected paths or copies used.
<code>outputs</code>	Expected run products.
<code>artifacts</code> / <code>typed_artifacts</code>	Typed artifacts for the UI.
<code>warnings</code>	Useful risks that do not invalidate the entire result.
<code>errors</code>	Clean failures or blocks, without raw tracebacks.
<code>qa</code>	Quality findings and metrics.
<code>provenance</code>	Original-input policy, environment, and source.
<code>next_actions</code>	A person's next step.
<code>original_modified</code>	<code>false</code> for normal routes.
<code>app_hints</code>	Short summary, visual severity, tags, and suggested previews.

4.1. Canonical statuses

Status	Meaning
PASS	The run finished and its result is usable.
WARNING	Useful output requires human review.
BLOCKED_CONTROLADO	A backend, input, platform, or safety condition prevents execution; the block is clean and actionable.
FAIL	Clean failure without products that appear valid.
ROTO	Audit classification for raw tracebacks, false success, misleading artifacts, or changed originals. It should not be a public tool's normal status.

5. Artifact types

Initial taxonomy:

Artifact type	Meaning
<code>summary_json</code>	App-parseable summary.
<code>manifest_json</code>	Provenance: inputs, outputs, command, and environment.
<code>report_md</code>	Human-readable Markdown report.
<code>preview_png</code>	Static visual preview.
<code>table_csv</code>	Derived table.
<code>notebook_ipynb</code>	Generated, inspected, or copy-executed notebook.
<code>log_txt</code>	stdout, stderr, command log, or transcript.

<code>qa_report</code>	Dedicated QA report.
<code>handoff_bundle</code>	Folder or package for continuation or delivery.
<code>unknown</code>	Existing artifact without a more specific type.

6. Run bundles

Reference: `references/v1-8-run-bundle-contract.md`. Target layout:

```
run/
manifest.json
summary.json
stdout.txt
stderr.txt
command.txt
artifacts/
previews/
reports/
tables/
logs/
next_steps.md
```

This contract lets Scientific Workbench discover results without fragile heuristics; it does not require rewriting every script in v1.8. Priority general routes already have focused bundle or wrapper coverage. Scientific and legacy routes can migrate gradually.

7. Dry-run router

References: `references/v1-8-workflow-router.md` and `scripts/scientific_workflow_router.py`.

Mode	Use
<code>inspect</code>	Inventories the input and proposes candidate routes.
<code>plan</code>	Returns dry-run steps and suggested commands without executing them.
<code>explain</code>	Explains why a capability applies, does not apply, or is unknown.

Example:

```
python scripts/scientific_workflow_router.py plan input.csv \
--task "profile an anonymized professional table" \
--summary-json run/router_summary.json \
--manifest-json run/router_manifest.json
```

The router does not run downstream tools or modify inputs. It should reject astronomy or legacy routes for general input.

8. Capability app-readiness

Reference: `references/v1-8-app-readiness-matrix.md`. Recorded v1.8 counts:

Class	Total	Use
<code>app_ready</code>	17	App invocation with sufficiently clear output.
<code>app_ready_partial</code>	18	Useful, but needs a wrapper, typing, documentation, or UX work.
<code>blocked_optional</code>	7	Optional backend or platform: preflight panel, not an ordinary action.
<code>cli_only</code>	9	Valid CLI route, but narrow or expert-oriented for a normal app flow.
<code>maintainer_only</code>	6	Release, registry, smoke, or maintenance only.
<code>not_applicable_to_app</code>	2	Unexposed unless a clear expert context applies.

8.1. Recommended exposure

- **Normal:** tables, documents, containers, mixed folders, time series, physical QA, and noise budgets.
- **Expert:** FITS, WCS, astrometry, sky crossmatching, RGB/FITS, RV, photometric calibration, spectroscopy, and advanced presentations.
- **Optional panel:** STILTS/TOPCAT, APT, DuckDB, iWork, Keynote, and TEAREDUCE.
- **Legacy expert:** IRAF, fxcor, iSTARMOD, and narrow coursework routes.
- **Maintainer:** smoke, synchronization, registry, capability probes, and gates.

9. Testing a capability as an app would

Minimum pattern:

1. Copy or synthesize input in a fresh `tmp/` run.
2. Use the dry-run router when the case is ambiguous.
3. Run the capability with `-summary-json` and, where available, `-manifest-json` or `-run-dir`.
4. Verify that `summary.json` parses.
5. Check `contract_version`, `app_status`, `typed_artifacts`, `original_modified=false`, `warnings`, `errors`, and `next_actions`.
6. Confirm that the original input has not changed.
7. Classify output as `PASS`, `WARNING`, `BLOCKED_CONTROLADO`, `FAIL`, or audit `ROTO`.

Examples:

```
python scripts/profile_table.py input.csv --run-dir tmp/run
python scripts/audit_v1_8_phase6_app_like_general_regression.py
```

For scientific routes:

```
python scripts/audit_v1_8_phase7_app_like_science_astro_regression.py
```

10. Optional, legacy, and domain-specific routes

10.1. Optional

Optional means that an external backend or platform is required, not that the route is broken. It must not become a core dependency. STILTS, APT, DuckDB, iWork, Keynote, and TEAREDUCE need preflight and a clear block when they do not apply.

10.2. Legacy

IRAF, fxcor, and iSTARMOD are real but narrow tools. They should not be the first normal-mode app action. Expert mode may block them or guide the user toward a safe copy.

10.3. Domain-specific

Not every pattern transfers. `li6708_equivalent_width_workbench.py` `measure` targets a particular spectral line, not general measurement. `catalog_workbench.py` `crossmatch-sky` transfers tolerance-based matching but remains sky-only.

11. Scientific Workbench

Reference: `references/v1-8-scientificworkbench-integration.md`. Compatibility interpretation:

- Scientific Workbench invokes local capabilities.
- AI can plan and explain; local capabilities execute.
- The app already uses run folders, summaries, manifests, and artifact discovery.
- v1.8 strengthens the backend to make this integration less fragile.

v1.8 prepares the skill; v2.0 will synchronize the app. This historical guide does not claim that Scientific Workbench consumes the entire v1.8 contract.

12. v1.8 regressions

Gate	Coverage
<code>audit_v1_8_app_ready_contract_regression.py</code>	JSON contract, examples, and legacy compatibility.
<code>audit_v1_8_app_readiness_matrix_regression.py</code>	59 rows, counts, and Top 10 P1 priorities.
<code>audit_v1_8_phase4_run_bundles_regression.py</code>	Run bundles and artifact types.
<code>audit_v1_8_phase5_router_regression.py</code>	Dry-run routing on six inputs.
<code>audit_v1_8_phase6_app_like_general_regression.py</code>	Tables, documents, notebooks, ZIP, time series, sensors, and a personal folder.
<code>audit_v1_8_phase7_app_like_science_astro_regression.py</code>	Science, astronomy, optional, and legacy routes.
<code>audit_v1_8_phase8_docs_regression.py</code>	Guide, references, PDF, and key terms.

13. Work reserved for v1.9

The historical plan for v1.9 is consolidation:

- Reduce P2 wrapper and artifact-typing debt.
- Stabilize envelope names and fields.
- Extend `-run-dir` to more priority routes.
- Strengthen regressions while retaining the same principles.
- Decide how to handle editable/installed differences before final synchronization.
- Prepare a stable matrix for app integration.

14. Work reserved for v2.0

The historical v2.0 scope can be larger:

- Synchronize Scientific Workbench with the mature skill backend.
- Consume `typed_artifacts`, `app_hints`, and `next_actions`.
- Use dry-run routing as a deterministic planning fallback.
- Separate normal, expert, optional, legacy, and maintainer modes in the UI.
- Align app run folders with the run bundle contract.
- Keep provenance, logs, manifests, and the read-only policy visible to users.

15. Historical Phase 8 checklist

- v1.8 references present.
- TeX guide and compiled PDF available.
- PDF text extracted and validated.
- No public-surface drift under `sync_public_surface_docs.py -check`.
- Scientific Workbench not edited.
- Installed skill not synchronized.